Lecture Indexer: Prototype Review

CS 6030 Cloud Computing — Semester Project, Milestone 2

Theodore Podewil
Dept. of Computer Science
Western Michigan University
Kalamazoo, MI, USA

Joshua Jubenville
Dept. of Computer Science
Western Michigan University
Kalamazoo, MI, USA

Andrew Wojciechowski
Dept. of Computer Science
Western Michigan University
Kalamazoo, MI, USA

TABLE I
MILESTONE 2 DELIVERABLES

Deliverable	How we met it
Initial deployment	Production environment on Google Cloud and Azure, provisioned with Terraform and running live
CI/CD pipeline	Automated tests on every change; one-approval releases with automatic rollback; three releases shipped
Basic functionality	Sign-in, lecture registration, search, and deep links, tested end to end in production

***Abstract*—Milestone 2 asks for an initial deployment, a CI/CD pipeline, and basic functionality. Lecture Indexer (LI) has all three. The system runs in production on Google Cloud, with Microsoft Azure as the second provider, and all of its infrastructure is defined in Terraform. A GitHub Actions pipeline tests every change and releases to production after one team approval. The core workflow from our proposal works end to end: a professor registers a YouTube lecture, and a student searches it and jumps to the moment a term is spoken.**

***Index Terms*—cloud-native, multi-cloud, Kubernetes, CI/CD, Terraform, GitHub Actions.**

I. Introduction

LI makes recorded lectures searchable. It retrieves the timed captions of a course's YouTube videos, indexes them, and returns links that open each video at the moment a search term is spoken [1]. This report covers the prototype we built for Milestone 2. Table I summarizes how we met each deliverable, and the following sections describe each one. Source code, workflow runs, and releases are public at https://github.com/SisyphusRex/cloud-computing-semester-project.

II. Architecture

Fig. 1 shows the system as deployed. A React web app, hosted on Cloud Run, calls two containerized Flask services on Google Kubernetes Engine (GKE): an Ingest API that registers lectures and a Search API that answers queries. Each service scales independently. They share a Cloud SQL for PostgreSQL database, whose built-in full-text search provides the index. Users sign in with their Google accounts. Secrets are kept in Secret Manager, and the services reach Google Cloud without stored keys.

Changes from the proposal. The proposal placed the web app on Azure Static Web Apps. Our student Azure subscription cannot create that service in any region it allows, so the web app moved to Cloud Run [2]. Azure remains our second provider: it holds the identities the pipeline uses, and it will store the nightly backups. We also made sign-in invite-only, with roles managed by admins inside the app [3].

III. Deliverable 1: Initial Deployment

All infrastructure is written in Terraform and can be rebuilt from the repository [4]. Terraform creates the network, the GKE cluster, the database, the container registry, the secrets, and the Cloud Run service on Google Cloud, plus the resource groups and identities on Azure. The Kubernetes configuration is versioned as well, including the autoscaling rules, network policies, and HTTPS ingress.

The environment has run in production since October 18, 2026. Figs. 2–4 show the services running on GKE and Cloud Run and the Azure identities. Because a student budget is limited, one command pauses the paid resources between test sessions without losing any data. This cuts the idle cost to a few dollars a month.

IV. Deliverable 2: CI/CD Pipeline

The pipeline runs on GitHub Actions in two stages [5]:

- **Continuous integration.** Every push and pull request runs linting and the automated tests, builds the container images, and checks the Terraform and Kubernetes configuration (Fig. 5).
- **Continuous delivery.** Every merge to `main` starts a release (Fig. 6). The pipeline first shows the changes and the infrastructure plan. A team member approves once, and the pipeline then updates the infrastructure, backs up the database if needed, builds and deploys the services and the web app, and tags the release. If any step fails, it rolls back automatically, and an earlier release can be restored at any time.

The pipeline stores no cloud passwords or keys. It signs in to Google Cloud and Azure with short-lived OIDC tokens.

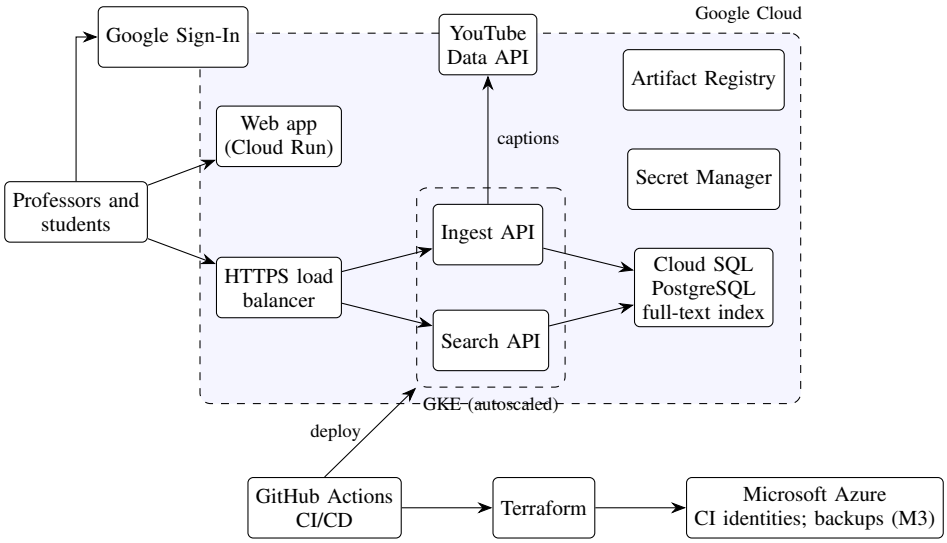


Fig. 1. The deployed architecture. The application runs on Google Cloud. Azure holds the CI identities and, from Milestone 3, the off-site database backups. GitHub Actions and Terraform deploy to and provision both clouds.

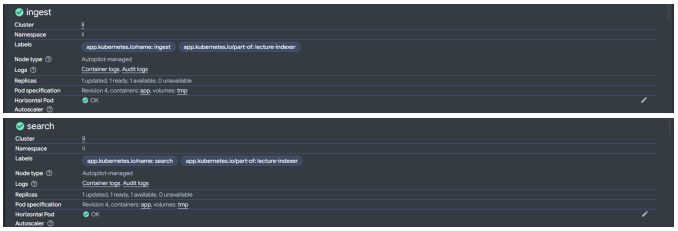


Fig. 2. The Ingest and Search services running on GKE, each healthy and autoscaled.

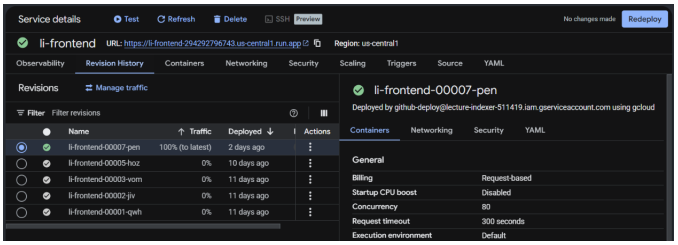


Fig. 3. The web app on Cloud Run: each release creates a new revision, and traffic moves to it only after a health check passes.

The pipeline has shipped three releases so far. The first one needed a few attempts, as we found and fixed configuration problems. Each failed attempt stopped safely and left the running system untouched, which is what the pipeline was designed to do.

V. Deliverable 3: Basic Functionality

The prototype supports the full workflow from our proposal:

1) Users sign in with Google. Only invited users get in (Fig. 7).
2) A professor creates a course and registers a lecture from their own YouTube channel (Fig. 8). LI downloads the captions, cleans them, and indexes them.

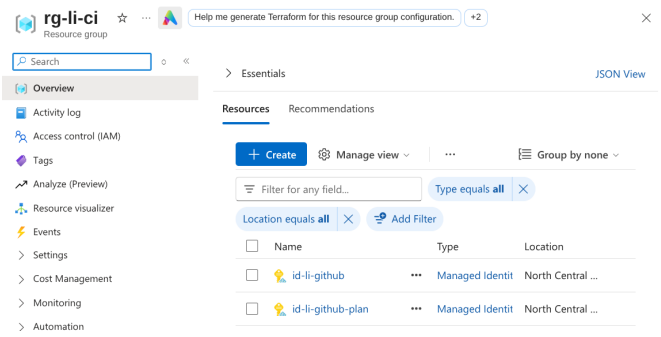


Fig. 4. Azure, the second cloud provider: the identities the release pipeline signs in as.

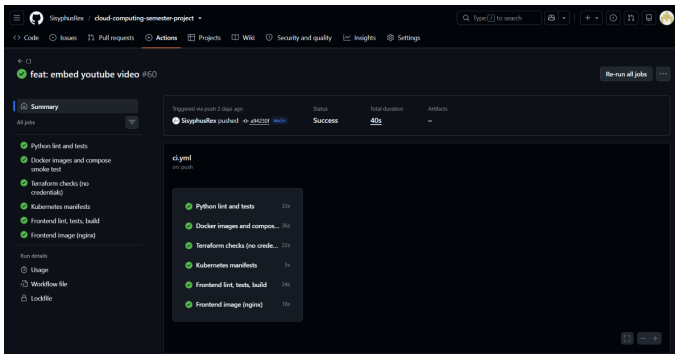


Fig. 5. A passing CI run: tests, images, and configuration checks.

3) A student searches the course and gets ranked, highlighted excerpts. Each excerpt plays the video at the matching moment (Fig. 9).
4) Admins invite users and assign roles (Fig. 10).

Testing. We tested the workflow with real Google and YouTube accounts, first locally and then in production [6].

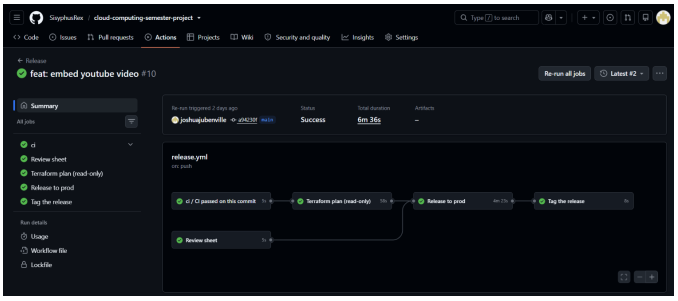


Fig. 6. A release run, from the change review and infrastructure plan through deployment and tagging.

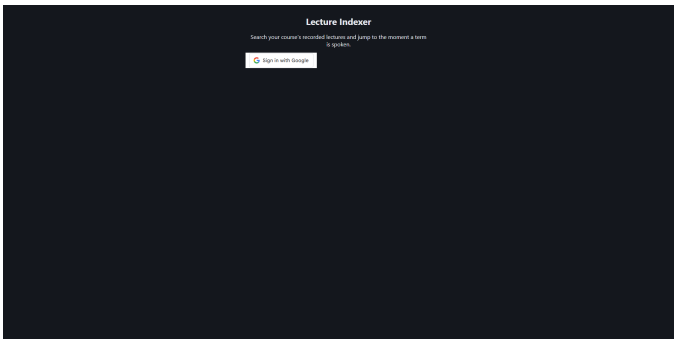


Fig. 7. Google sign-in.

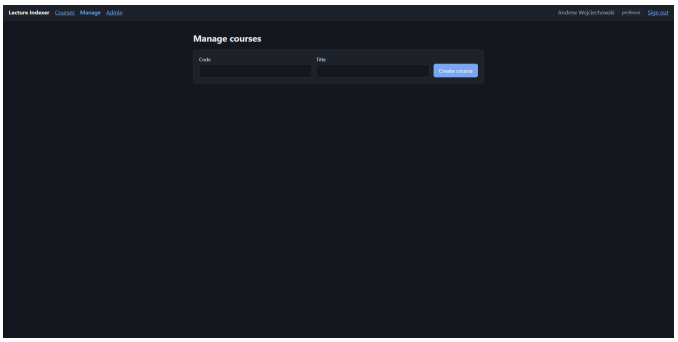


Fig. 8. The Manage page, where professors create courses and register lectures.

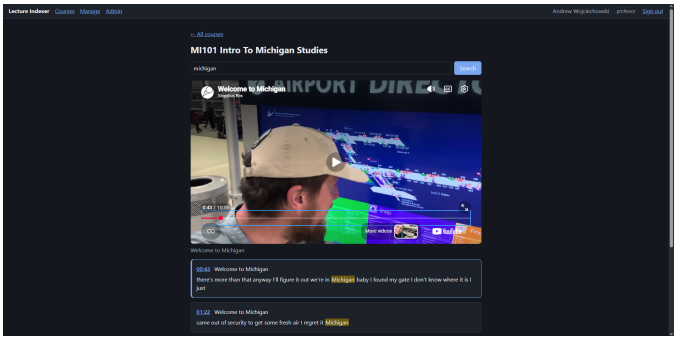


Fig. 9. Search results with highlighted excerpts and the embedded player at the matching moment.

The tests covered registering a video, searching it as a student,

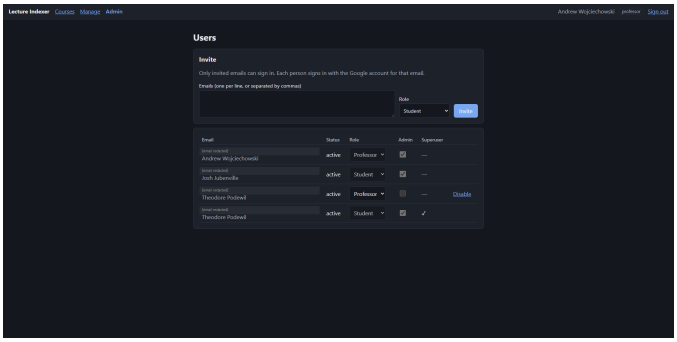


Fig. 10. The Admin page for invitations and roles (email addresses redacted).

re-registering and deleting lectures, and confirming that a professor cannot register someone else's video. Over 120 automated tests run on every change.

VI. Next Steps

Milestone 3 adds the remaining pieces from the proposal: nightly encrypted database backups to Azure, a monitoring dashboard with alerts, and a written security review.

References

[1] T. Podewil, J. Jubenville, and A. Wojciechowski, "Lecture Indexer: A multi-cloud service for timestamped keyword search of recorded lectures," CS 6030 Milestone 1 proposal, Western Michigan Univ., 2026.

[2] "ADR 0001: Host the frontend on Cloud Run instead of Azure Static Web Apps," https://github.com/SisyphusRex/cloud-computing-semester-project/blob/main/docs/architecture_decision_records/0001-frontend-on-cloud-run.md

[3] "ADR 0002: Invite-only sign-in, with roles set by admins in the database," https://github.com/SisyphusRex/cloud-computing-semester-project/blob/main/docs/architecture_decision_records/0002-invite-only-users-and-admin-roles.md

[4] "Bootstrap runbook" and "First deployment," https://github.com/SisyphusRex/cloud-computing-semester-project/tree/main/docs

[5] "Release process," https://github.com/SisyphusRex/cloud-computing-semester-project/blob/main/docs/release-process.md; releases: https://github.com/SisyphusRex/cloud-computing-semester-project/releases

[6] "Real Google/YouTube test: progress log," https://github.com/SisyphusRex/cloud-computing-semester-project/blob/main/docs/milestones/m2_prototype_review/real-youtube-test.md

APPENDIX. AI USAGE

TABLE II
AI TOOL USAGE FOR THIS REPORT AND MILESTONE 2

Item	**Details**
AI tool used	Claude (Claude Code)
Purpose	1. To plan a report that maps each Milestone 2 deliverable to evidence in our repository 2. To draft the report in IEEE format and build the PDF 3. To redact email addresses and account details from the screenshots
Example prompt	*"Make a plan for creating a report showing how we have met the deliverables for milestone 2."*
Generated output used	This report, *CS6030_F26_PrototypeReview_LectureIndexer.pdf*
Verification performed	We checked the report's facts against our deployment notes, test logs, and GitHub release records, took the screenshots ourselves from the live system, and revised the draft until it matched what we built.
AI use toward Milestone 2	We built the prototype with Claude Code in four logged sessions (October 13–19, 2026). Claude proposed the repository layout and implementation plan; wrote most of the services, Terraform, Kubernetes configuration, CI/CD workflows, and frontend; helped run the real Google/YouTube test; and diagnosed the failed first release attempts. About two thirds of the commits are co-authored by Claude. The team chose the framework and design decisions, approved every plan before Claude built it, made all merges and release approvals, and verified the result with the automated tests and end-to-end testing in production. Each prompt, with a summary of Claude's response, is logged in `docs/ai_usage/`.